CASTILLO ENGINEERING | PLANSET QC PLATFORM

Project Overview

AI-assisted QA/QC for solar PV plansets

Document	1 of 9
Audience	All stakeholders
Revision	Rev 1.0 · 2026
Status	Handoff

Castillo Planset QC — Project Overview

1. What the tool is and the problem it solves

Castillo Planset QC is an AI-assisted quality-assurance / quality-control (QA/QC) application for solar photovoltaic (PV) plansets. It ingests an engineering drawing set as a PDF and produces a triaged, evidence-backed QC report that a reviewer can open, adjust, and export to Excel.

Reviewing a solar planset by hand is slow, repetitive, and error-prone. A single set can run to dozens of sheets, and a competent check has to confirm that every required sheet exists and is in order, that NEC electrical math (string voltages, ampacity, conductor and grounding sizing, clearances) is correct, that title blocks and keywords are present, and — the classic defect — that the *same value* agrees across every sheet it appears on. Castillo Planset QC automates the mechanical portion of that work so the human reviewer spends their time on judgment, not on transcription.

The tool combines three complementary engines and reconciles their output into one checklist:

1. **Deterministic NEC electrical calculations** — exact, reproducible math (no AI) for the numeric rules.
2. **AI vision review** — per-page and multi-page model calls that read values, labels, and dimension callouts directly off the drawings.
3. **A rules engine** — a YAML-driven catalog of roughly 400 rules across 20+ sheet categories, plus keyword, sheet-existence, and title-block checks.

Every finding carries a **status** (Pass / Fail / Needs Review / Deferred), a **severity**, a **confidence** score, an **evidence** excerpt, and a **page/location** pointer. Findings are de-duplicated across engines before they reach the reviewer.

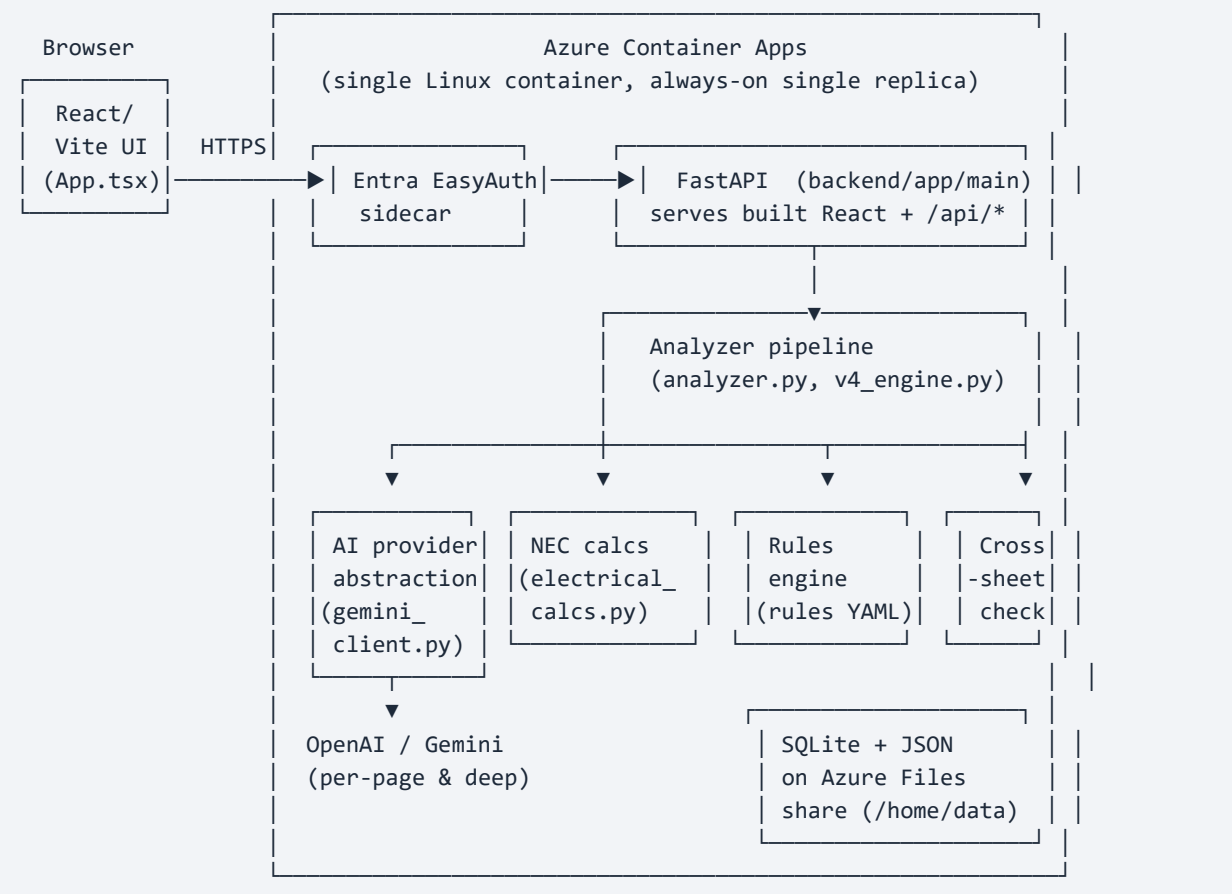
2. Who uses it

Role	How they use it
QC Engineer	Primary user. Uploads plansets, picks the design stage, reviews the category list and issue cards, overrides or adds findings, and exports the Excel checklist.
Engineer of Record (PE)	Consumes the QC output as part of stamping/approval, and is the sign-off authority for engineering-judgment changes to the calculation rules (e.g. the pending MCOV change — see Current Status).
Developer	Maintains the FastAPI backend, the React frontend, the rule catalog, and the AI provider abstraction.

DevOps / Automation Engineer	Owns the Azure Container Apps deployment, CI/CD workflows, Entra sign-in, and the persistent Azure Files share.

Access is restricted to the organization: production sign-in is single-tenant **Microsoft Entra** (castillope.com org login) enforced by the Azure Container Apps built-in EasyAuth sidecar, which injects a trustworthy signed-in identity used for run attribution.

3. High-level architecture



Request flow, in words:

4. The **React/Vite frontend** (`frontend/src/App.tsx`) talks to a single-origin **FastAPI** app (`backend/app/main.py`). The same container serves the built static UI and the `/api/*` routes.
5. FastAPI accepts an uploaded PDF at `POST /api/analyze`, registers the work in a **bounded job queue** (`jobs.py`), and drives the **analyzer pipeline** (`analyzer.py` orchestrating `v4_engine.py`, `electrical_calcs.py`, `consistency.py`, `diagram_consistency.py`, `supporting_docs.py`).

6. The pipeline fans out to three engines: **deterministic NEC calcs**, the **rules engine** (loaded from `rules_v4_draft.yaml` / `stage_overrides.yaml` via `rule_registry.py`), and **AI vision/text** review through the provider abstraction in `gemini_client.py`.
7. Findings are merged, de-duplicated, assigned a status/severity/confidence/evidence, and persisted to **SQLite plus JSON sidecars** and rendered artifacts (cropped snippets, highlighted page previews).
8. The frontend polls progress and job status, renders the triaged report, and offers **XLSX export** via `GET /api/export/{run_id}`.

AI provider abstraction: `gemini_client.py` exposes provider-neutral functions (`analyze_page_image`, `analyze_multiple_images`, `analyze_text`, `analyze_document`) and selects the backend via the `AI_PROVIDER` env var. Both **OpenAI** and **Google Gemini** are supported. Each provider distinguishes a fast **per-page/standard** model from a **deep** model for heavier reasoning. Optional `OPENAI_SEED` requests best-effort run-to-run determinism from the OpenAI chat-completion API.

4. Technology stack

Backend (backend/requirements.txt, Python 3.11+)

Component	Package	Version
Web framework	`fastapi`	0.116.1
ASGI server	`uvicorn[standard]`	0.35.0
Multipart uploads	`python-multipart`	0.0.20
PDF rendering/parsing	`PyMuPDF` (fitz)	1.26.4
PDF text/table extraction	`pdfplumber`	0.11.7
Excel export	`openpyxl`	3.1.5
Image handling	`Pillow`	11.3.0
Gemini SDK	`google-genai`	1.14.0
OpenAI SDK	`openai`	≥ 1.30.0
Config / env	`python-dotenv`	1.1.0
Rules loading	`PyYAML`	6.0.2

Frontend (frontend/package.json)

Component	Package	Version
UI library	`react` / `react-dom`	^18.3.1
Build tool / dev server	`vite`	^5.4.10

React plugin	`@vitejs/plugin-react`	^4.3.1
Language	`typescript`	^5.6.2
Type defs	`@types/react`, `@types/react-dom`	^18.3.x

Frontend scripts: `npm run dev` (Vite dev server, default `http://127.0.0.1:5173`), `npm run build` (`tsc -b && vite build`), `npm run preview`.

AI models (production configuration, infra/main.bicep)

Setting	Value
`AI_PROVIDER`	`openai`
`OPENAI_MODEL` (per-page / standard)	`gpt-5.4-mini`
`OPENAI_MODEL_DEEP` (deep reasoning)	`gpt-5.4`

> Note: the *code defaults* in `gemini_client.py` are more conservative (`AI_PROVIDER=openai`, `OPENAI_MODEL=gpt-4.1-mini`, `GEMINI_MODEL=gemini-2.5-flash`); production overrides them to the `gpt-5.4` family via container environment variables. When temperature is not exposed (the `gpt-5.x` reasoning models), determinism is pursued only through the optional `OPENAI_SEED`.

Hosting & platform

- **Azure Container Apps** — single Linux container (FastAPI serves the built React UI), one always-on replica.
- **Azure Files share** mounted at `/home/data` — persistent SQLite database, uploaded PDFs, generated snippets/previews, and exported workbooks.
- **Azure Container Registry** `castilloqaqcautomationacr` — image builds, pulled via **managed identity**.
- **Microsoft Entra** single-tenant EasyAuth for org-only sign-in.
- **GitHub Actions** CI/CD via OIDC (`az acr build` + `az containerapp update`).

5. Major capabilities

- **Projects** — runs are organized under named projects. The sidebar groups Project → Design Stage → lineage (a rerun chain keyed by `root_run_id`) → versions. Backed by `GET /api/projects`, `GET /api/projects/{project_id}`.
- **Rerun version history** — re-analysis is **non-destructive**; each rerun (`POST /api/runs/{run_id}/reanalyze`) is saved as a new version so prior results are never lost. History is served by `GET /api/runs/{run_id}/versions`.

- **Design-stage gating** — the reviewer selects a stage (**30%**, **60% / IFP**, **90%**, **IFC**, or **As-Built**). Rules whose `min_stage` is later than the selected stage are **Deferred** — *unless the target sheet is actually present in the PDF*, in which case it is still checked. A cross-reference filter also defers rules that name sheets or external documents absent from the upload. Thresholds live in `stage_overrides.yaml`.
- **Bounded run queue + shared Activity feed** — an in-memory `ThreadPoolExecutor` (`jobs.py`) runs at most `ANALYZE_CONCURRENCY` analyses at once (default **2**); the rest wait as `queued`. A shared **Activity panel** in the UI shows every in-flight analysis across all signed-in engineers with live progress, and raises **completion notifications** (optional desktop alerts). Job state is exposed by `GET /api/jobs`.
- **Cross-sheet consistency** — flags the same engineering quantity (transformer kVA, voltages, module/string counts, wire sizes, GCR, pole/pile spacing, row pitch, equipment tags, …) disagreeing between two sheets. A structured, unit-tolerant comparator (`consistency.py`) is fed by a vision pass (`diagram_consistency.py`) that reads values off the drawings, an AI reconciler for ambiguous free-text, and a **same-referent verification** step so two genuinely different quantities are never mistakenly compared. Confirmed conflicts are **Fail**; any doubt stays **Needs Review**. Stored as multi-location findings and exported with a `Locations` column.
- **Supporting-document / tech-spec cross-checks** — optional engineering documents (module / inverter / transformer datasheets, CESIR, PVSyst, ampacity tables, relay/protection studies, structural calcs, owner BOD / tech spec) are parsed (`supporting_docs.py`) and made available as evidence. Datasheet specs **auto-merge into project details** to fill gaps, but user-entered values always win. PDF, XLSX, and DOCX inputs are supported. Endpoints: `POST /api/parse-project-details`, `POST /api/parse-supporting-docs`.
- **Deterministic NEC calcs** — string Voc(cold)/Vmp(cold)/Vmp(hot) per NEC 690.7, system-info math (module/DC/AC totals, DC/AC ratio), multi-transformer sizing, fuse and ampacity (DC/AC/MV), EGC/GEC sizing, voltage drop, and NEC 110.26 clearances.
- **XLSX export** — `exporter.py` builds a formatted workbook with a summary sheet and per-category counts across **Pass / Fail / Needs Review / Deferred / Overridden**, plus a due-diligence template. Served by `GET /api/export/{run_id}` and `GET /api/due-diligence-template`.
- **Cross-stage compare** — a stage-aware compare picker and stage-labeled diff banner let the reviewer diff one run against another (including across design stages) to see how findings changed between revisions.
- **Signed-in attribution** — `GET /api/me` (backed by `auth.py`) resolves the Entra identity from EasyAuth headers so each run records who started it; locally, `DEV_USER_EMAIL` stands in.

Status color convention: Pass = green, Fail = red, Needs Review = amber, Deferred = purple (rendered as N/A with a precise "Requires X" reason).

6. Environments at a glance

	Production	Sandbox
Container App	`castillo-qaqc-automation`	`castillo-qaqc-sandbox`
Resource group	`castillo-qaqc-automation-rg`	(sandbox environment)
Deploys from branch	`master`	`staging`
Image registry	`castilloqaqcautomationacr` (managed-identity pull)	same ACR
Sign-in	Microsoft Entra single-tenant (castillope.com) EasyAuth	Entra EasyAuth
Persistence	SQLite + artifacts on Azure Files share at `/home/data`	separate share
AI models	`gpt-5.4-mini` (per-page) / `gpt-5.4` (deep)	configurable per env
CI/CD	GitHub Actions, OIDC → `az acr build` + `az containerapp update`	same, staging trigger

Both run as a single always-on replica. The full runbook is in `DEPLOYMENT.md`; container build and infrastructure are in `Dockerfile` and `infra/main.bicep`.

7. Current status

- **Wave-1 QC-feedback remediation is in an open pull request**, pending sign-off from the **Engineer of Record (PE)** on the **MCOV (maximum continuous operating voltage) change**. That single calculation-rule change is an engineering-judgment call and is held for PE approval before merge; the rest of the wave is ready.
- **NEC-edition selector is designed but not yet built.** The intended feature lets an engineer choose the applicable NEC code year (2017 / 2020 / 2023 / 2026), pinned per project, so calculations and rules follow the correct edition. The design exists; no implementation has shipped.
- Additional roadmap items (multi-file project linking with revision diffs, role-based access separation, PDF report export alongside XLSX, deeper PVSyst side-by-side comparison, expanded E-200 and civil/structural rule coverage, and a packaged Windows executable) remain future work.